Карта языка Go

всё, что нужно новичку — на одном листе

программа · значения · строки · наборы · логика · память

Волнистые линии идут от центра к веткам. **Соседние ветки связаны между собой** — читай по кругу, начиная с любой.

Программа

с чего всё начинается

```
package main

func main() {
}
```

> Запускается только пакет main с функцией main
> Другое имя пакета — библиотека, сама не запускается
> Имя с большой буквы видно снаружи пакета, с маленькой — нет
> `go mod init имя` заводит модуль в папке
> `import` подключает чужой пакет: `fmt` печать, `strings` строки, `strconv` число↔строка, `os` аргументы
> Комментарии: `//` до конца строки, `/* */` блоком

Сборка

как текст становится программой

`исходник .go` → компилятор → `бинарник`, файл из машинного кода.

> Ошибки ловятся до запуска: не собралось — не работает
> `go run .` собрал во временную папку, запустил, стёр
> `go build .` оставил файл рядом с кодом
> Бинарник ни от чего не зависит, но собран под свою систему
> `gofmt -w .` — отступы, `go vet .` — проверка

Переменные

имя, за которым стоит значение

```
x := 10         // создать
x = 20          // изменить
var z int       // z == 0
const Pi = 3.14
```

> `:=` создаёт новую, работает только внутри функции
> `=` пишет в уже существующую
> Переменная живёт внутри своих скобок и исчезает за ними
> Создал и не использовал — ошибка сборки, а не замечание

Типы

и нулевые значения

> `int` целое · ноль `0` · `7/2` даёт `3`
> `float64` дробное · `0` · точное не всегда
> `string` текст · `""`
> `bool` да-нет · `false`
> `byte` 0…255 · `rune` один символ
> слайс, map, указатель · `nil`

Типы не смешиваются сами: `int + float64` не соберётся, нужно `float64(x)`.

Строки и руны

то, на чём спотыкаются чаще всего

строка "Привет" в памяти — это байты:

208	159	209	128	208	184	208	178	208	181	209	130
П		р		и		в		е		т	

len = 12 байтов, но букв 6 — на кириллицу уходит по 2 байта

> `len(s)` считает байты, а не буквы
> `s[i]` отдаёт байт — число, а не букву
> `for _, r := range s` идёт по символам, r это руна
> `[]rune(s)` переводит в символы, тогда `len` даст 6
> Строку изменить нельзя: `s[0]='H'` не соберётся, только собрать новую
> `'a'` это символ-число 97, `"a"` это строка. `'9'-'0'` даёт `9`
> `s[2:5]` — кусок строки с 2-го по 4-й байт

GO

Компилируемый язык со строгими типами. Одна программа — один бинарник.

Массивы и слайсы

главная пара, которую путают

слайс — это окно в массив, а не сами данные:

1	2	3	4	5	6	7	8

len = 3

cap = 6

len — сколько видно · cap — сколько влезет до переезда в новый массив

> `[3]int` — массив: длина вшита в тип, `[3]int` и `[5]int` разные типы, копируется целиком
> `[]int` — слайс: длина растёт через `append`, копируется только окно
> `s = append(s, x)` — присваивать обязательно, иначе результат теряется
> `b := a` копирует окно, а не данные — запись в b изменит a. Настоящая копия только через `copy`
> Двумерный: `[][]int` — слайс слайсов, каждый ряд создаётся отдельно

Условия

развилки

```
if n > 0 && n < 10 {
} else {
}
switch c {
case 'a', 'e':
default:
}
```

> `&&` — верны оба · `||` — хотя бы одно · `!` — наоборот
> Сравнение: `==` `≠` `<` `>` `⩽` `⩾`
> В `switch` ветка сама останавливается, `break` не нужен
> Можно объявить переменную прямо в условии: `if v, ok := m[k]; ok {}` — она живёт только внутри `if`
> Диапазон пишется двумя условиями: `n ⩾ '0' && n ⩽ '9'`

Циклы

в Go он один — for

```
for i := 0; i < 10; i++ {}
for n > 0 {}
for {}
for i, v := range s {}
```

> Слов while и do в языке нет вообще
> `break` выходит из цикла, `continue` — к следующему шагу
> Цикл внутри цикла — так рисуют таблицы и фигуры: внешний считает строки, внутренний — столбцы

```
for row := 0; row < h; row++ {
    for col := 0; col < w; col++ {
    }
    fmt.Println()   // конец строки
}
```

> Границу проверяй на первом и последнем шаге: `i < len(s)`, а не `⩽`

Функции

приняли → вернули

```
func Sum(a, b int) int {
    return a + b
}
func Div(a, b int) (int, error)
```

> Печать ≠ возврат. `Println` показывает человеку, `return` отдаёт программе
> `return` прерывает функцию сразу
> Рекурсия — функция зовёт саму себя. Обязательно нужно условие остановки, иначе программа падает
> Функцию можно передать как значение: `f func(int) int`
> `nums ...int` — сколько угодно аргументов, внутри это слайс
> Возвраты можно назвать заранее: `func f() (n int, err error)`
> Имя с большой буквы делает функцию видимой другим пакетам

Память

копия или ссылка



> Копией идут: число, строка, `bool`, массив, `struct`
> Ссылкой идут: указатель, слайс, map
> `&x` взять адрес · `*T` тип адреса · `nil` адрес в никуда
> `struct` — свой тип из полей. Функция с `(u *User)` перед именем становится методом

Ввод и вывод

`os.Args` аргументы запуска, `[0]` — имя программы
`Println` печать + перевод строки
`Sprintf` возвращает строку, не печатает
`bufio.Scanner` читает ввод построчно
`Printf` печать по шаблону
`%d %s %v %q %T %c` число, строка, любое, в кавычках, тип, символ

Мар — доступ по ключу

`make(map[string]int)` создать; без этого запись обрушит программу
`m["a"] = 1` положить или заменить
`v, ok := m["x"]` ok говорит, лежал ли ключ вообще
`delete(m, "a")` убрать пару
`range` порядок случайный — Go мешает нарочно

Ошибки и словарь

`n, err := ...` ошибка приходит вторым значением, проверяй сразу
`if err ≠ nil` nil значит ошибки не было. try/catch в Go нет

`пакет` папка с файлами под одним именем
`бинарник` готовая программа одним файлом
`срез` окно в массив, длина меняется
`компиляция` перевод текста в машинный код
`руна` один символ любого языка
`указатель` адрес вместо самого значения